

```

rm(list = ls())

# Local github
codepath <- c("/media/larryleon/My Projects/GitHub/Forest-Search/R/")
source(paste0(codepath, "source_forestsearch_v0.R"))
source_fs_functions(file_loc = codepath)

library(kableExtra)
library(knitr)
library(ggplot2)
library(gridExtra)
library(cubature)
library(aVirtualTwins)
library(randomForest)
library(survival)
library(survminer)
library(grf)
library(policytree)
library(data.table)
library(plyr)
library(dplyr)
library(glmnet)
library(corrplot)

library(table1)

maxFollow <- 84
cens.type <- "weibull"

# m1 -censoring adjustment
muC.adj <- log(1.5)
k.z3 <- 1
k.treat <- 0.9

z1_frac <- 0.25 # Default model index 'm1' (The 1st quartile of z1=er)
pH_super <- 0.125 # non-NULL re-defines z1_frac

if (is.null(pH_super)) {
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_frac)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_frac)))
  cat("Underlying pH_super", c(pH_check), "\n")
}
# pH_super specified If pH_super then override z1_frac and find z1_frac to
# yield pH_super

if (!is.null(pH_super)) {
  # Approximate Z1 quantile to yield pH proportion
  z1_q <- uniroot(propH.obj4, c(0, 1), tol = 1e-04, pH.target = pH_super)$root
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_q)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_q)))
  cat("pH", c(pH_check), "\n")
  rel_error <- (pH_super - pH_check)/pH_super
  if (abs(rel_error) >= 0.1)

```

```

    stop("pH_super approximation relative error exceeds 10%")
    z1_frac <- z1_q
    cat("Underlying pH_super", c(pH_check), "\n")
  }

## pH 0.122449
## Underlying pH_super 0.122449

# Bootstrap on log(hr) scale converted to HR (est.loghr=TRUE & est.scale='hr')
t.start.all <- proc.time()[3]

##### Forest search criteria

hr.threshold <- 1.25 # Initital candidates
hr.consistency <- 1 # Candidates for many splits
pconsistency.threshold <- 0.9
stop.threshold <- 0.95
maxk <- 2
nmin.fs <- 60
pstop_futile <- 0.5
# Limit timing for forestsearch
max.minutes <- 3
m1.threshold <- Inf # Turning this off (Default)
# pconsistency.threshold<-0.70 # Minimum threshold (will choose max among
# subgroups satisfying)
fs.splits <- 400 # How many times to split for consistency
# vi is % factor is selected in cross-validation --> higher more important
vi.grf.min <- 0.2
# Null, turns off grf screening
d.min <- 10 # Min number of events for both arms (d0.min=d1.min=d.min)
# default=5 Virtual twins analysis Counter-factual difference (C-E) >=
# vt.threshold Large values in favor of C (control)
vt.threshold <- 0.225 # For VT delta
treat.threshold <- 0

maxdepth <- 2
n.min <- 60
ntree <- 1000
# GRF criteria
dmin.grf <- 12 # For GRF delta
# Note: For CRT this represents dmin.grf/2 RMS for control (-dmin.grf/2 for
# treatment)
frac.tau <- 0.6

outcome.name <- c("y.sim")
event.name <- c("event.sim")
id.name <- c("id")
treat.name <- c("treat")

cox.formula.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat"))
cox.formula.adj.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5"))

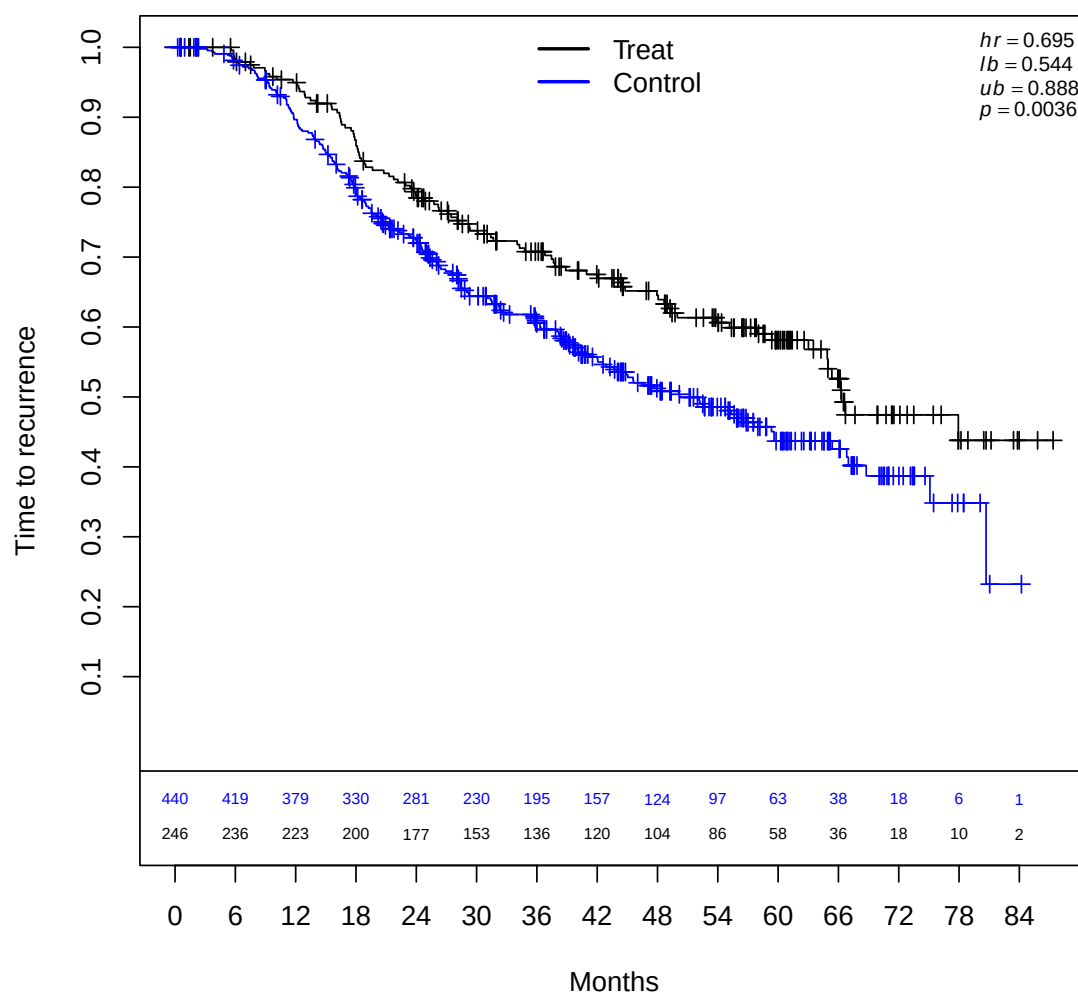
mod.harm <- "alt"
hrH.target <- 2
# out.loc = NULL turns off file creation

```

```

N <- 1000
this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, hrH.target = hrH.target,
  cens.type = cens.type, out.loc = NULL, details = TRUE, parms_torand = FALSE)

```



```

## Super-population empirical harm and non-harm hazard ratios= 2.000007 0.6466405
## Causal HR (empirical ITT)= 0.7057463

```

```

file_out <- c("output/sim-FS4_N1k_hrH=2_sim1.Rdata")

```

```

# sim sensH ppvH sensHc ppvHc lasso_omit 1: 78 0.4402985 0.5412844 0.9422633
# 0.9158249 1 2: 164 0.6146789 0.5275591 0.9326599 0.9518900 1 3: 221 0.3149606
# 0.5063291 0.9553265 0.9055375 1 4: 257 0.5528455 0.5666667 0.9407070
# 0.9375000 1 5: 273 0.5422535 0.5065789 0.9125874 0.9233491 1

```

```

n_add_noise <- 3

```

```

sim <- 1

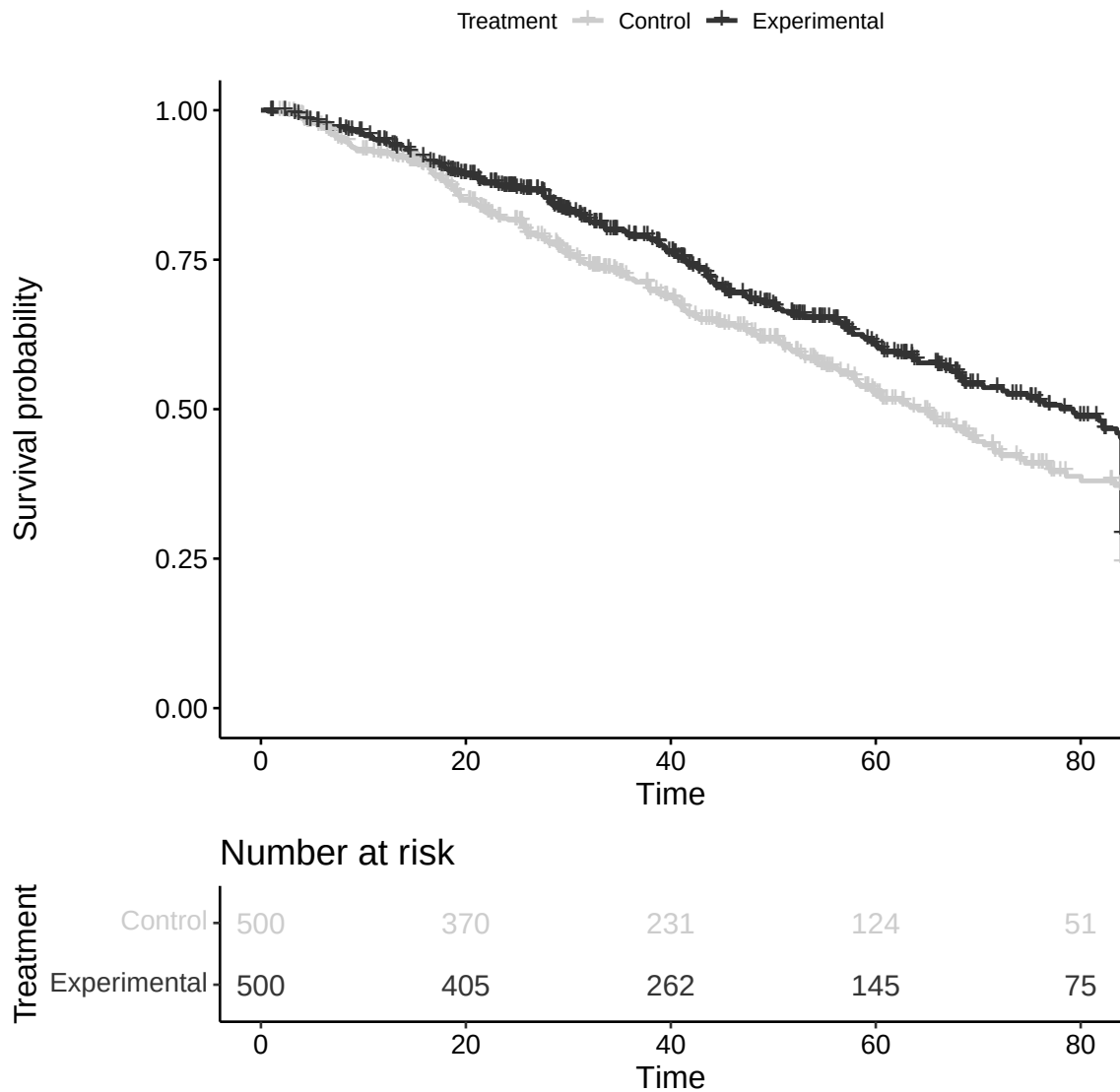
```

```
dgm <- this.dgm$dgm

x <- sim_aftm4_gbsg(dgm = dgm, n = N, maxFollow = maxFollow, muC.adj = muC.adj, simid = sim)

kmfit <- survfit(Surv(y.sim, event) ~ treat, data = x)

ggsurvplot(kmfit, data = x, main = "K-M curves for simulated data", legend = "top",
  legend.title = "Treatment", legend.labs = c("Control", "Experimental"), palette = "grey",
  risk.table = TRUE, risk.table.col = "strata")
```



```
if (n_add_noise == 0) {
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3")
}

if (n_add_noise == 5) {
  set.seed(8316951 + 1000 * sim)
  # Add 5 noise
  x$noise1 <- rnorm(N)
```

```

x$noise2 <- rnorm(N)
x$noise3 <- rnorm(N)
x$noise4 <- rnorm(N)
x$noise5 <- rnorm(N)
confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3", "noise1",
  "noise2", "noise3", "noise4", "noise5")
}

if (n_add_noise == 3) {
  set.seed(8316951 + 1000 * sim)
  # Add 3 noise
  x$noise1 <- rnorm(N, sd = 1)
  x$noise2 <- rnorm(N, sd = 1)
  x$noise3 <- rnorm(N, sd = 1)
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3", "noise1",
    "noise2", "noise3")
}

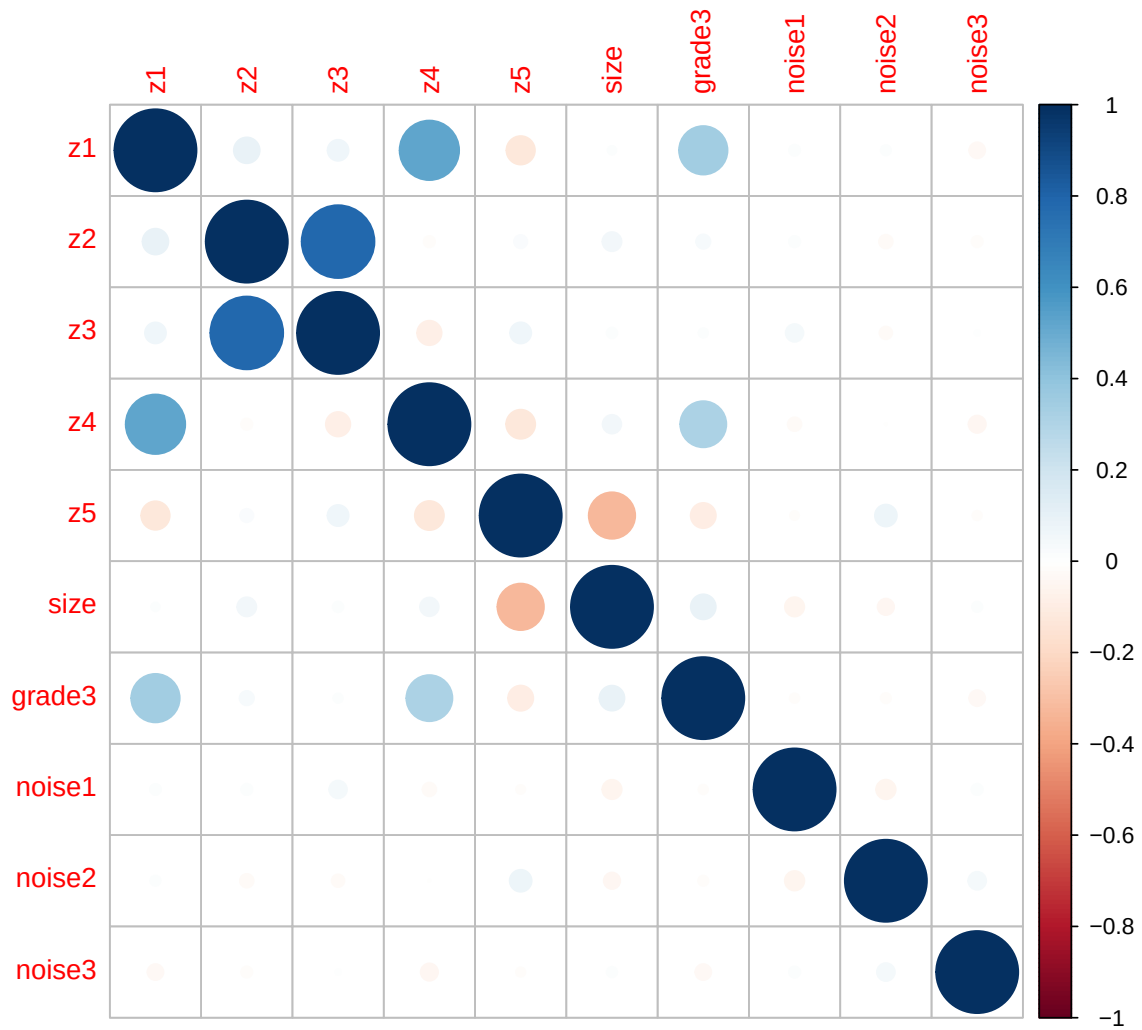
# More challenging which_replace <- which(confounders.name=='z1')
# confounders.name[which_replace] <- 'er'

cox.formula.check <- as.formula(paste("Surv(y.sim,event.sim)~treat+z1+z2+z3+z4+z5+size+grade3+noise1",
  "noise2", "noise3"))
coxph(cox.formula.check, data = x)

## Call:
## coxph(formula = cox.formula.check, data = x)
##
##              coef exp(coef)    se(coef)      z        p
## treat  -0.3025913  0.7389010  0.0883067 -3.427 0.000611
## z1      0.1661221  1.1807173  0.1128285  1.472 0.140929
## z2     -0.3511168  0.7039015  0.1471411 -2.386 0.017021
## z3      0.2335106  1.2630262  0.1502286  1.554 0.120096
## z4      0.6227676  1.8640799  0.1053275  5.913 3.37e-09
## z5     -0.9356975  0.3923121  0.0956500 -9.783 < 2e-16
## size   -0.0009214  0.9990791  0.0029355 -0.314 0.753619
## grade3  0.1520049  1.1641660  0.1036108  1.467 0.142355
## noise1  0.0116625  1.0117308  0.0449186  0.260 0.795144
## noise2 -0.0333165  0.9672324  0.0429412 -0.776 0.437830
## noise3 -0.0044072  0.9956025  0.0409940 -0.108 0.914385
##
## Likelihood ratio test=223.5 on 11 df, p=< 2.2e-16
## n= 1000, number of events= 533

Zm <- cor(as.matrix(x[, c(confounders.name)]))
corrplot(Zm)

```



```
suppressWarnings(table1(~z1 + z2 + z3 + z4 + z5 + size + grade3 + noise1 + noise2 +
  noise3 | treat, data = x))
```

	0	1	Overall
	(N=500)	(N=500)	(N=1000)
z1			
Mean (SD)	0.262 (0.440)	0.254 (0.436)	0.258 (0.438)
Median [Min, Max]	0 [0, 1.00]	0 [0, 1.00]	0 [0, 1.00]
z2			
Mean (SD)	0.518 (0.500)	0.546 (0.498)	0.532 (0.499)
Median [Min, Max]	1.00 [0, 1.00]	1.00 [0, 1.00]	1.00 [0, 1.00]
z3			
Mean (SD)	0.426 (0.495)	0.440 (0.497)	0.433 (0.496)
Median [Min, Max]	0 [0, 1.00]	0 [0, 1.00]	0 [0, 1.00]
z4			
Mean (SD)	0.474 (0.500)	0.450 (0.498)	0.462 (0.499)
Median [Min, Max]	0 [0, 1.00]	0 [0, 1.00]	0 [0, 1.00]
z5			
Mean (SD)	0.520 (0.500)	0.568 (0.496)	0.544 (0.498)
Median [Min, Max]	1.00 [0, 1.00]	1.00 [0, 1.00]	1.00 [0, 1.00]
size			
Mean (SD)	29.2 (13.9)	30.2 (15.2)	29.7 (14.6)
Median [Min, Max]	25.5 [4.00, 120]	26.0 [3.00, 120]	26.0 [3.00, 120]
grade3			
Mean (SD)	0.264 (0.441)	0.226 (0.419)	0.245 (0.430)
Median [Min, Max]	0 [0, 1.00]	0 [0, 1.00]	0 [0, 1.00]
noise1			
Mean (SD)	-0.00497 (0.971)	-0.0151 (0.970)	-0.0100 (0.970)
Median [Min, Max]	-0.0231 [-3.05, 2.80]	0.0127 [-2.93, 2.79]	-0.0181 [-3.05, 2.80]
noise2			
Mean (SD)	-0.00535 (1.02)	0.0489 (1.00)	0.0218 (1.01)
Median [Min, Max]	0.0129 [-2.80, 3.19]	0.0864 [-2.82, 3.54]	0.0414 [-2.82, 3.54]
noise3			
Mean (SD)	0.0235 (1.01)	0.0248 (1.02)	0.0242 (1.01)
Median [Min, Max]	0.0961 [-3.03, 2.92]	0.0166 [-3.05, 2.37]	0.0556 [-3.05, 2.92]

```
# Options Allconfounders.name is list of confounders within analysis dataset
# (1) use_lasso=TRUE & use_grf=FALSE Lasso used to possibly reduce dimension
# Any continuous factors are cut at medians (2) use_lasso=TRUE & use_grf=TRUE
# Lasso used to reduce dimension Continuous covariates are cut at medians
# However, if GRF selects a covariate cut then only that cut is used: For
# example if 'age <= median(ag)' is called for per Lasso but GRF includes 'age
# <= 54', then only the latter is used (3) use_grf_only = TRUE (overrides
# use_lasso and use_grf) Only factors selected via GRF are used (4) use_lasso =
# F & use_grf = T Median cuts (unless selected via GRF) as in (2) However no
# possible dimension reduction via lasso All categorical factors included (5)
# If all options set to false, then all factors are included with continuous
# factors cut at medians
```

```
use_lasso <- TRUE
use_grf <- TRUE
use_grf_only <- FALSE
```

```
fs.est <- forestsearch(df.analysis = x, Allconfounders.name = confounders.name, details = TRUE,
  use_lasso = use_lasso, use_grf = use_grf, use_grf_only = use_grf_only, dmin.grf = 12,
```

```

frac.tau = 0.6, conf_force = NULL, outcome.name = outcome.name, treat.name = treat.name,
event.name = event.name, id.name = id.name, n.min = nmin.fs, hr.threshold = hr.threshold,
hr.consistency = hr.consistency, fs.splits = fs.splits, d0.min = d.min, d1.min = d.min,
pstop_futile = pstop_futile, pconsistency.threshold = pconsistency.threshold,
stop.threshold = stop.threshold, max.minutes = max.minutes, maxk = maxk, by.risk = 12,
plot.sg = TRUE, vi.grf.min = vi.grf.min)

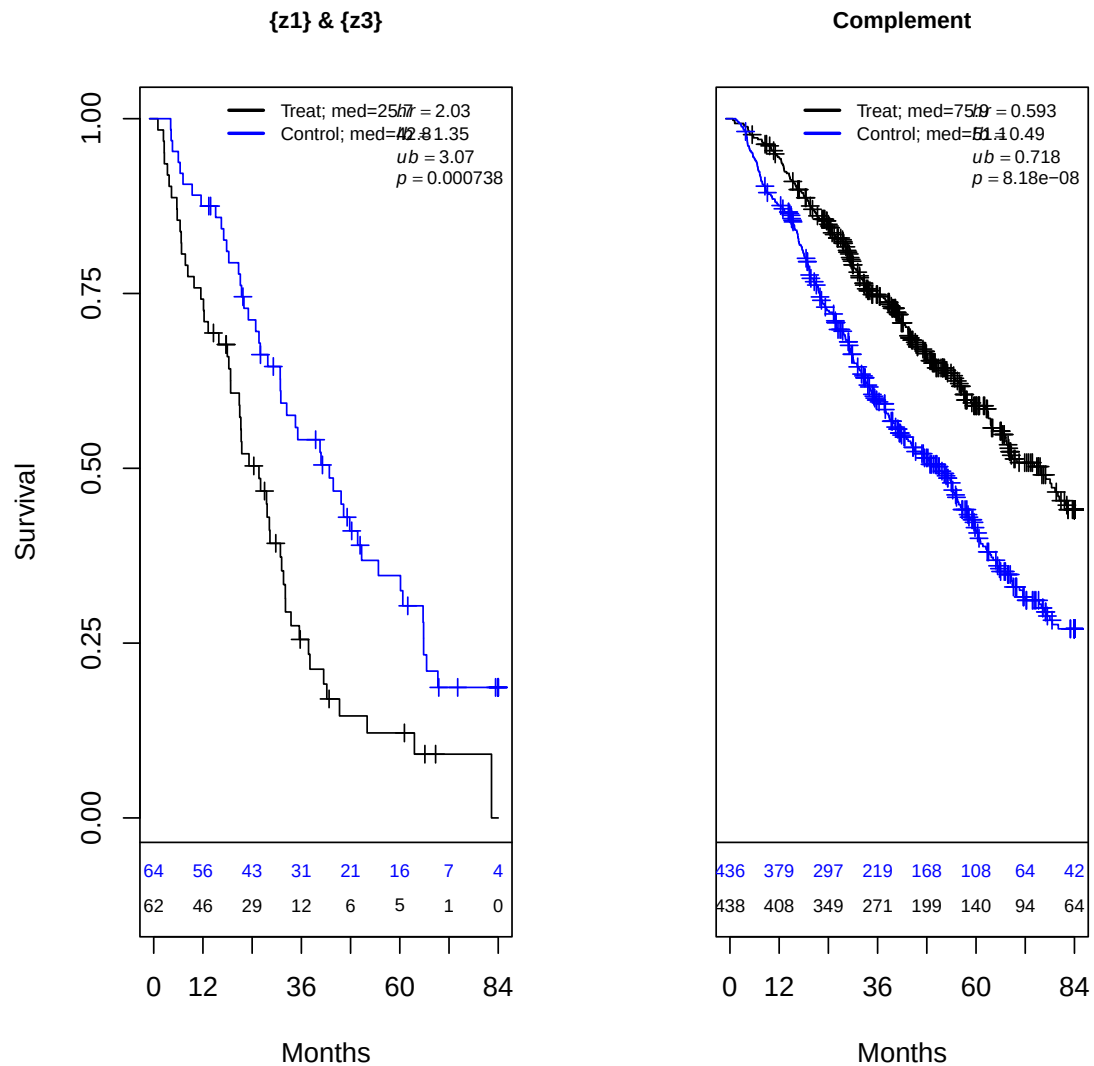
## tau, maxdepth= 48.03382 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2   -4.7123385   742.0000000   0.9990188    4.7123385   742.0000000
## 2         3    3.2680174   258.0000000   1.9379845   -3.2680174   258.0000000
## 3         4   -6.8489605   382.0000000   1.4593663    6.8489605   382.0000000
## 4         5    2.4637676    86.0000000   3.0459817   -2.4637676    86.0000000
## 5         6   -4.0476219   373.0000000   1.3590584    4.0476219   373.0000000
## 6         7    7.9294041   159.0000000   2.4303007   -7.9294041   159.0000000
##   treated.se      diff Nsg depth
## 1  0.9990188  -9.424677  742     1
## 2  1.9379845   6.536035  258     1
## 3  1.4593663 -13.697921  382     2
## 4  3.0459817   4.927535   86     2
## 5  1.3590584  -8.095244  373     2
## 6  2.4303007  15.858808  159     2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 6         7    7.929404   159.000000   2.430301   -7.929404   159.000000
##   treated.se      diff Nsg depth
## 6  2.430301  15.85881  159     2
## Subgroup found
## [1] "z2 <= 0"          "noise3 <= 1.02" "z1 <= 0"
## [1] "z1 <= 0"
## # of continuous/categorical characteristics 4 6
## Continuous characteristics: size noise1 noise2 noise3
## Categorical characteristics: z1 z2 z3 z4 z5 grade3
## CV lambda = 0.006192456
## 10 x 1 sparse Matrix of class "dgCMatrix"
##           s0
## z1      0.1915920655
## z2     -0.2361194324
## z3      0.1317510575
## z4      0.5899499477
## z5     -0.9178029865
## size   -0.0005293987
## grade3  0.1444283045
## noise1  .
## noise2 -0.0298588432
## noise3  .
## Cox-LASSO selected: z1 z2 z3 z4 z5 size grade3 noise2
## Cox-LASSO not selected: noise1 noise3
## Median cuts after Lasso size noise2
## Categorical after Lasso z1 z2 z3 z4 z5 grade3
## Factors prior to removing if also in GRF: size noise2
## Factors after removing any duplicates also in GRF: size noise2
## Initial GRF cuts included z2 <= 0 noise3 <= 1.02 z1 <= 0
## Factors included per GRF (not in lasso) noise3 <= 1.02
## # of candidate subgroup factors= 9
## [1] "size <= median(size)"      "noise2 <= median(noise2)"
## [3] "noise3 <= 1.02"          "z1"

```

```

## [5] "z2"                "z3"
## [7] "z4"                "z5"
## [9] "grade3"
## Confounders per grf screening q4 q5 q6 q2 q1 q8 q7 q3 q9
## FSconfounders.name      vi.cs  vi_ratio
## 4      q4 0.38882519 3.4994267
## 5      q5 0.14879424 1.3391482
## 6      q6 0.09645729 0.8681156
## 2      q2 0.07007577 0.6306819
## 1      q1 0.06985324 0.6286791
## 8      q8 0.06091639 0.5482475
## 7      q7 0.05837040 0.5253336
## 3      q3 0.05518586 0.4966727
## 9      q9 0.05152163 0.4636946
## Number of unique levels (L) and possible subgroups= 18 262143
## # of subgroups based on # variables > k.max and excluded (per million) 0.261972
## k.max= 2
## Events criteria for control,exp= 10 10
## # of subgroups with events less than criteria: control, experimental 11 12
## # of subgroups with sample size less than criteria 13
## # of subgroups meeting all criteria = 157
## # of subgroups fitted (Cox model estimable) = 157
## *Subgroup Searching Minutes=* 0.0303
## Number of subgroups meeting HR threshold 7
## # of candidate subgroups (meeting HR criteria) = 7
## Subgroups (1st 10) meeting overall screening thresholds (HR, m1) sorted by HRs= Inf
##      K   n   E d1    m1    m0   HR L(HR) U(HR) q4.0 q4.1 q5.0 q5.1 q6.0 q6.1
## 1:  2 126  96 51 25.66 42.82 2.03  1.35  3.07   0   1   0   0   0   1
## 2:  2 159 118 64 27.65 45.69 1.67  1.16  2.41   0   1   0   1   0   0
## 3:  2 129  88 49 25.66 47.43 1.66  1.09  2.54   0   1   0   0   0   0
## 4:  2 144 121 60 21.33 30.99 1.53  1.06  2.20   0   1   0   0   0   0
## 5:  2 127  93 51 25.66 40.79 1.42  0.94  2.13   0   1   0   0   0   0
## 6:  2 129  93 48 31.19 35.12 1.30  0.86  1.96   0   1   0   0   0   0
## 7:  2 220 154 78 30.89 35.12 1.28  0.93  1.76   0   1   0   0   0   0
## 8: NA  NA  NA NA    NA    NA   NA   NA   NA   NA  NA  NA  NA  NA  NA
## 9: NA  NA  NA NA    NA    NA   NA   NA   NA   NA  NA  NA  NA  NA  NA
## 10: NA  NA  NA NA    NA    NA   NA   NA   NA   NA  NA  NA  NA  NA  NA
##      q2.0 q2.1 q1.0 q1.1 q8.0 q8.1 q7.0 q7.1 q3.0 q3.1 q9.0 q9.1
## 1:    0    0    0    0    0    0    0    0    0    0    0    0
## 2:    0    0    0    0    0    0    0    0    0    0    0    0
## 3:    0    1    0    0    0    0    0    0    0    0    0    0
## 4:    0    0    0    0    1    0    0    0    0    0    0    0
## 5:    0    0    1    0    0    0    0    0    0    0    0    0
## 6:    0    0    0    0    0    0    0    0    0    0    0    1
## 7:    0    0    0    0    0    0    0    0    0    1    0    0
## 8:   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
## 9:   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
## 10:  NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
## Consistency 1
## # of splits= 400
## Model, % Consistency Met= {z1} {z3} 1
## Number of subgroups meeting consistency criteria=
##      p.consistency Nsg group.id m.index K  M.1  M.2
## 1:      1 126      2      1 2 {z1} {z3}

```



```
## [1] "{z1}" "{z3}"
## % consistency criteria met= 1
## SG focus= hr
## Subgroup Consistency Minutes= 0.02623333
## Subgroup found
## Minutes overall= 0.08613333

grf.est <- grf.subg.harm.survival(data = x, confounders.name = confounders.name,
  outcome.name = outcome.name, event.name = event.name, id.name = id.name, treat.name = treat.name,
  n.min = n.min, dmin.grf = dmin.grf, frac.tau = 0.6, details = TRUE)

## tau, maxdepth= 48.03382 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2  -4.7123385   742.0000000  0.9990188    4.7123385   742.0000000
## 2         3   3.2680174   258.0000000  1.9379845   -3.2680174   258.0000000
## 3         4  -6.8489605   382.0000000  1.4593663    6.8489605   382.0000000
## 4         5   2.4637676    86.0000000  3.0459817   -2.4637676    86.0000000
## 5         6  -4.0476219   373.0000000  1.3590584    4.0476219   373.0000000
## 6         7   7.9294041   159.0000000  2.4303007   -7.9294041   159.0000000
```

```
##      treated.se      diff Nsg depth
## 1  0.9990188 -9.424677 742      1
## 2  1.9379845  6.536035 258      1
## 3  1.4593663 -13.697921 382      2
## 4  3.0459817  4.927535  86      2
## 5  1.3590584 -8.095244 373      2
## 6  2.4303007 15.858808 159      2
##      leaf.node control.mean control.size control.se treated.mean treated.size
## 6          7      7.929404    159.000000    2.430301    -7.929404    159.000000
##      treated.se      diff Nsg depth
## 6  2.430301 15.85881 159      2
## Subgroup found
## [1] "z2 <= 0"          "noise3 <= 1.02" "z1 <= 0"
## [1] "z1 <= 0"

# plot(grf.estftree)
```

```
library(doParallel)
registerDoParallel(parallel::detectCores(logical = FALSE))

cox.formula.boot <- cox.formula.sim
max.minutes <- 7
# Suggest running 50, first ... to get timing estimate

NB <- 1000

df_boot_analysis <- fs.est$df.est

fitH <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 0), cox.formula = cox.formula
H_obs <- fitH$est_obs # log(hr) scale
seH_obs <- fitH$se_obs
# Hc observed estimates
fitHc <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 1), cox.formula = cox.formul
Hc_obs <- fitHc$est_obs
seHc_obs <- fitHc$se_obs
rm("fitH", "fitHc")

Ystar_mat <- bootYstar({
  ystar <- get_Ystar(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
# Check dimension
if (dim(Ystar_mat)[1] != NB | dim(Ystar_mat)[2] != nrow(df_boot_analysis)) stop("Dimension of Ystar_

tB.start <- proc.time()[3]
# Bootstraps
resB <- bootPar({
  ans <- fsboot_forparallel(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
tB.now <- proc.time()[3]
tB.min <- (tB.now - tB.start)/60

doParallel::stopImplicitCluster()

cat("Minutes for Boots", c(NB, tB.min), "\n")

## Minutes for Boots 1000 3.390083
```

```

cat("Projection per 1000", c(tB.min * (1000/NB)), "\n")
## Projection per 1000 3.390083

cat("Propn bootstrap subgroups found =", c(sum(!is.na(resB$H_biasadj_1))/NB), "\n")
## Propn bootstrap subgroups found = 0.975

# How many timed out
cat("Number timed out=", c(sum(is.na(resB$H_biasadj_1) & resB$tmmins_search > max.minutes)),
    "\n")
## Number timed out= 0

H_estimates <- get_dfRes(Hobs = H_obs, seHobs = seH_obs, H1_adj = resB$H_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)
Hc_estimates <- get_dfRes(Hobs = Hc_obs, seHobs = seHc_obs, H1_adj = resB$Hc_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)

print(H_estimates)

##           HO          sdHO HO_lower HO_upper          H1          sdH1 H1_lower H1_upper
## 1: 2.031472 0.4265966 1.346058 3.065899 1.749058 0.4553779 1.049999 2.913531

print(Hc_estimates)

##           HO          sdHO HO_lower HO_upper          H1          sdH1 H1_lower
## 1: 0.5934173 0.05774354 0.4903797 0.7181049 0.6058823 0.09023169 0.4525037
##           H1_upper
## 1: 0.8112494

# H_estimates<-get_dfRes(Hobs=H_obs,seHobs=seH_obs,H1_adj=resB$H_biasadj_1,ystar=Ystar_mat,cov_metho
# Hc_estimates<-get_dfRes(Hobs=Hc_obs,seHobs=seHc_obs,H1_adj=resB$Hc_biasadj_1,ystar=Ystar_mat,cov_m
# print(H_estimates) print(Hc_estimates)

bootit <- list(H_estimates = H_estimates, Hc_estimates = Hc_estimates)
ans3 <- fsBoot.parallel.out(df = df_boot_analysis, FSboots = bootit, cox.formula.boot = cox.formula.
    sim_number = sim)
tall.min <- (tB.now - t.start.all)/60
cat("Overall minutes for analysis", c(tall.min), "\n")
## Overall minutes for analysis 3.555033

save(dgm, x, fs.est, grf.est, bootit, ans3, tall.min, resB, cox.formula.boot, file = file_out)

# ITT causal hazard ratio
mean(fs.est$df.est$hlin.ratio)

## [1] 0.8113636

# H Oracle
dfH <- subset(fs.est$df.est, flag.harm == 1)
mean(dfH$hlin.ratio)

## [1] 2.249393

dfH_est <- subset(fs.est$df.est, treat.recommend == 0)
mean(dfH_est$hlin.ratio)

## [1] 2.249393

```

Table 1 Simulated data example: Cox hazard ratio (hr) estimates for the ITT population and subgroups H and H^c . Cox model estimates are based on subgroups: H true (knowing the actual subgroup, a-priori); the estimated subgroup $\hat{H}$; and the bootstrap (1,000 resamples) bias-correction to $\hat{H}$ estimates, denoted $\hat{H}_{bc}$. Estimates for the complement H^c are defined analogously. The number of subjects in each population (# Subjects) and the number of subjects correctly classified (# Correct) in the true subgroups H and H^c , respectively, are listed.

	HR Estimate	Lower	Upper	# Subjects	# Correct
ITT	0.693	0.584	0.823	1000	.
H subgroup estimates					
H_{oracle}	2.031	1.345	3.067	126	.
$\hat{H}$	2.031	1.345	3.067	126	126
$\hat{H}_{bc}$	1.749	1.05	2.912	126	126
H-complement subgroup estimates					
H_{oracle}^c	0.593	0.49	0.718	874	.
$\hat{H}^c$	0.593	0.49	0.718	874	874
$\hat{H}_{bc}^c$	0.606	0.453	0.811	874	874

```
# Hc Oracle
dfHc <- subset(fs.est$df.est, flag.harm == 0)
mean(dfHc$hlin.ratio)

## [1] 0.6040505

dfHc_est <- subset(fs.est$df.est, treat.recommend == 1)
mean(dfHc_est$hlin.ratio)

## [1] 0.6040505

# dfSens <- subset(fs.est$df.est, flag.harm==1 & treat.recommend==0) sensH <-
# nrow(dfSens)/nrow(dfH)

## Confounders evaluated in Forestsearch:
## [1] "size <= median(size)"      "noise2 <= median(noise2)"
## [3] "noise3 <= 1.02"            "z1"
## [5] "z2"                         "z3"
## [7] "z4"                         "z5"
## [9] "grade3"
```